

GamutX

Loops

Python Basics



Introduction to Loops

- What is loop?
- The Purpose of Looping
- Types of Loops in Python

What is a Loop?

A loop is a way to **repeat a set of instructions without writing the same code over and over again.**

“Let’s say you want to **print all the numbers in a list.**”

```
numbers = [1, 2, 3, 4, 5]
```

```
print(numbers[0])  
print(numbers[1])  
print(numbers[2])  
print(numbers[3])  
print(numbers[4])
```

```
...  
# Output  
1  
2  
3  
4  
5  
...
```

● This takes forever

with loops

```
numbers = [1, 2, 3, 4, 5]
```

```
for n in numbers:  
    print(n)
```

```
...  
# Output  
1  
2  
3  
4  
5  
...
```

with a loop, we can write a few lines of code,
and Python will repeat them for us

Control flow statements

```
graph TD; A[Control flow statements] --> B[Conditional Statements]; A --> C[Transfer Statements]; A --> D[Iterative Statements];
```

Conditional Statements

- if Statement
- if-else Statement
- if-elif-else Statement
- Nested if Statement

Transfer Statements

- Break
- continue
- pass

Iterative Statements

- for
- while

Types of Loops in Python

- In Python, there are **three types of loops**

- **for loop**

- Used when you know **how many times you want to repeat something**.

- **while loop**

- Used when we want to **repeat something as long as a condition is true**.

- **Nested loops**

- A loop **inside another loop**.
 - Can be **for inside for, while inside while**, or a **mix**.

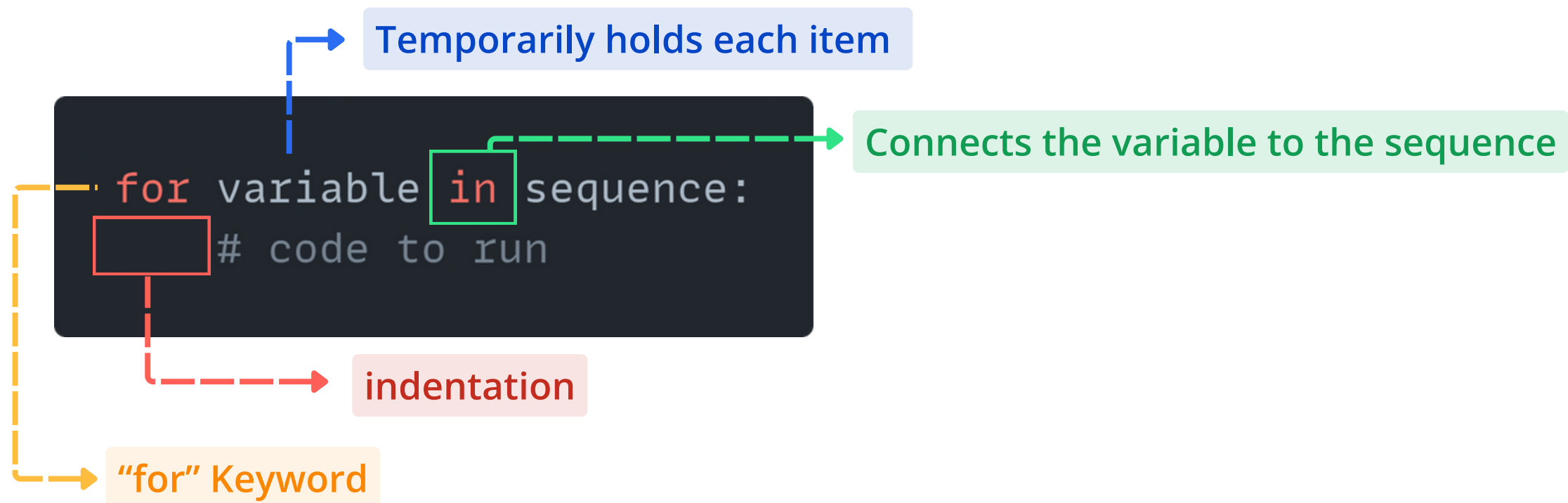
for Loop in Python

- Basic Syntax
- Iterating Through a String
- Iterating Through a List
- Iterating Through a Dictionary

Basic Syntax

- The **for loop** is **used when you want to repeat a block of code for each item in a sequence** — like a
 - **string** → "hello", "cat", "shark"
 - **list** → [1, 2, 3, 4], ["apple", "banana", "orange"]
 - **dictionary.** → { "name" : "rajesh" , "age" : 29 }

Basic Syntax



Iterating Through a String

```
word = "Python"
for char in word:
    print(char)
```



```
# P
# y
# t
# h
# o
# n
```

What's Happening Here?

- Python loops through each character in the string "Python". On each turn:
 - char stores the current character
 - print(char) displays it
- Loop runs once per letter — and then stops when it reaches the end.

Iteration Breakdown

Iteration 1:

char = 'P' **Output: P**

Iteration 2:

char = 'y' **Output: y**

Iteration 3:

char = 't' **Output: t**

Iteration 4:

char = 'h' **Output: h**

Iteration 5:

char = 'o' **Output: O**

Iteration 6:

char = 'n' **Output: n**

Concept Practice Task

Counting Characters in a String

```
sentence = "hello world"

count = 0

for char in sentence:
    count += 1 # Increment the counter each char

print("Total number of characters:", count)
#Output: 11
```

Extracting Vowels from a String

```
sentence = "hello world"

vowels = ""

for char in sentence:
    if char in 'aeiouAEIOU':
        vowels += char

print("Vowels in the string:", vowels)

#Output : eoo
```

Counting Specific Characters

```
sentence = "hello world"

target_char = 'o'

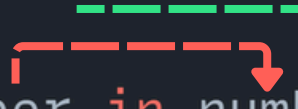
count = 0

for char in sentence:
    if char == target_char:
        count += 1

print(f"Number of '{target_char}' characters:", count)
# Output : Number of 'o' characters: 2
```


Iterating Through a List

```
numbers = [1, 2, 3, 4, 5]
for number in numbers:
    print(number)
```



```
# 1
# 2
# 3
# 4
# 5
```

What's Happening Here?

- Python goes through each item in the list.
- Each item is stored in the variable number.
- Then print(number) shows it.

Iteration Breakdown

Iteration 1:

number = 1 **Output: 1**

Iteration 2:

number = 2 **Output: 2**

Iteration 3:

number = 3 **Output: 3**

Iteration 4:

number = 4 **Output: 4**

Iteration 5:

number = 5 **Output: 5**

Concept Practice Task

Summing Elements in a List

```
numbers = [10, 20, 30, 40, 50]

total_sum = 0

for number in numbers:
    total_sum += number

print("Sum of all numbers:", total_sum)
# Output: Sum of all numbers: 150
```

Filtering Even Numbers from a List

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

even_numbers = []

for number in numbers:
    if number % 2 == 0:
        even_numbers.append(number)

print("Even numbers:", even_numbers)

# Output: Even numbers: [2, 4, 6, 8, 10]
```

Iterating Through a Dictionary

```
student_grades = {  
    'Alice': 85,  
    'Bob': 92,  
    'Charlie': 78  
}  
  
for student in student_grades:  
    print(student)
```

```
# Alice  
# Bob  
# Charlie
```

- If we want to **access both Key-Value Pairs**
- we need to **use item() methods**

What's Happening Here?

- When you **loop through a dictionary** like this, Python **goes through the keys**.
- **Each key (student name) is stored in student**, and printed.

Iteration Breakdown

Iteration 1:

student = 'Alice'

Output: Alice

Iteration 2:

student = 'Bob'

Output: Bob

Iteration 3:

student = 'Charlie'

Output: Charlie

Concept Practice Task

Iterating Over Dictionary Values

```
student_grades = {'Alice': 85, 'Bob': 92, 'Charlie': 78}

for grade in student_grades.values():
    print(grade)
```

```
# Output:
# 85
# 92
# 78
```

Iterating Over Dictionary Key-Value Pairs

```
student_grades = {'Alice': 85, 'Bob': 92, 'Charlie': 78}

for student, grade in student_grades.items():
    print(f"{student}: {grade}")
```

```
# Output:
# Alice: 85
# Bob: 92
# Charlie: 78
```

Filtering and Modifying Values in a Dictionary

```
student_grades = {'Alice': 85, 'Bob': 92, 'Charlie': 78}

high_achievers = {}

for student, grade in student_grades.items():
    if grade >= 85:
        high_achievers[student] = grade

print("High achievers:", high_achievers)
```

```
# Output: High achievers: {'Alice': 85, 'Bob': 92}
```

Iterating Through Keys - Values Using .items()

```
student_grades = {  
    'Alice': 85,  
    'Bob': 92,  
    'Charlie': 78  
}  
  
for name, grade in student_grades.items():  
    print(name, grade)
```

```
# Alice 85  
# Bob 92  
# Charlie 78
```

Key

Values

Method

What's Happening Here?

- .items() returns each (key, value) pair
- **name** stores the key → ('Alice', 'Bob'...)
- **grade** stores the value → (85, 92...)

Iteration Breakdown

Iteration 1:

- name = 'Alice'
- grade = 85
- **Output: Alice 85**

Iteration 2:

- name = 'Bob'
- grade = 92
- **Output: Bob 92**

Iteration 3:

- name = 'Charlie'
- grade = 78
- **Output: Charlie 78**

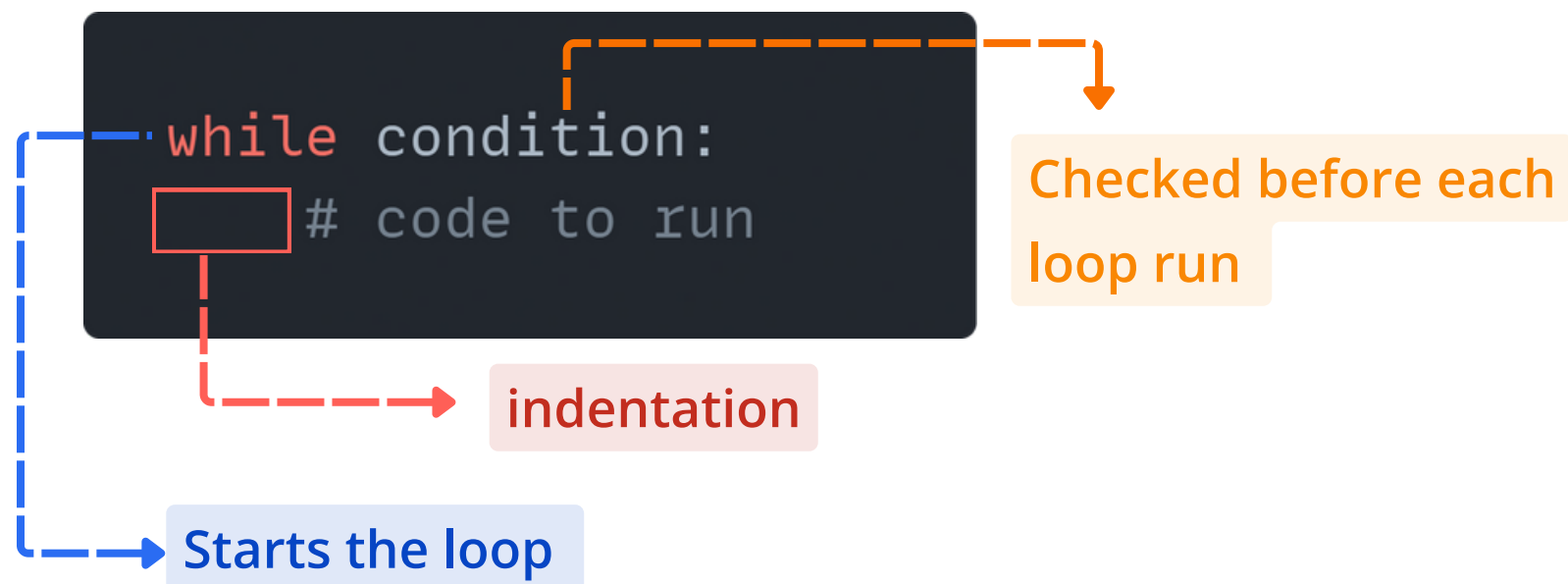
while Loop

- Basic Syntax
- Understanding - While Loop

Basic Syntax

- A while loop lets you **repeat a block of code** as long as a **condition is True**.
- It's **useful** when you **don't know in advance how many times something should repeat**
- the loop just **keeps running until the condition becomes False**.

Basic Syntax



Understanding - While Loop

Infinite Loop

```
while True:
    print("This loop will run forever...")
```

Condition stays True

Output

```
This loop will run forever...
This loop will run forever...
This loop will run forever...
...
```

What's Happening Here?

- **True** is always true — it never changes
- There's **no condition update** inside the loop to make it **false**
- So the loop runs **endlessly** unless you manually stop it (e.g., **Ctrl + C**)

Note:

- While **infinite loops** can be **useful in specific scenarios**, such as:
 - Running background services
 - Keeping web servers active
 - Waiting continuously for user input

Basic Counter with while Loop

```
i = 0
```

```
while i < 5:
```

```
    print(i)
```

```
    i += 1
```

Used to
break loop

```
# 0
```

```
# 1
```

```
# 2
```

```
# 3
```

```
# 4
```

How it Works

- We start with $i = 0$
- The condition $i < 5$ is True, so the loop runs
- It prints i , then increases it using $i += 1$
- This repeats until i becomes 5
- Once $i < 5$ is False, the loop stops

Iteration Breakdown

Iteration 1:

- $i = 0$
- **Output: 0**

Iteration 2:

- $i = 1$
- **Output: 1**

Iteration 3

- $i = 2$
- **Output: 2**

Iteration 4:

- $i = 3$
- **Output: 3**

Iteration 5:

- $i = 4$
- **Output: 4**

Iteration 6:

- $i = 5 \rightarrow i < 5 \rightarrow$ **False**
- **Loop ends**

Concept Practice Task

GamutX

Using while Loop for Summing Numbers

```
num = 1
total_sum = 0

while num <= 10:
    total_sum += num
    num += 1

print("Sum of numbers from 1 to 10 is:", total_sum)
```

Output:Sum of numbers from 1 to 10 is: 55

Using a while Loop to Validate User

```
user_input = -1

while user_input <= 0:
    user_input = int(input("Enter a positive number: "))

print("You entered:", user_input)
```

Output:
Enter a positive number: 34
You entered: 34

Concept Practice Task

GamutX

Iterating Over a String

```
text = "Hello"

index = 0

while index < len(text):
    print(text[index])
    index += 1

print("All characters printed.")
```

```
# Output:
# H
# e
# l
# l
# o
# All characters printed.
```

Iterating Over Integers

```
start = 1
end = 5

current = start

while current <= end:
    print(current)
    current += 1

print("All integers printed.")
```

```
# Output:
# 1
# 2
# 3
# 4
# 5
# All integers printed.
```

Iterating Over a List

```
numbers = [10, 20, 30, 40, 50]

index = 0

while index < len(numbers):
    print(numbers[index])
    index += 1

print("All numbers printed.")
```

```
# Output:
# 10
# 20
# 30
# 40
# 50
# All numbers printed.
```


Concept Practice Task

Iterating Over a Dictionary's Keys

```
grades = {"Alice": 85, "Bob": 90, "Charlie": 78}

keys = list(grades.keys())

index = 0

while index < len(keys):
    key = keys[index]
    print(key)
    index += 1

print("All keys printed.")
```

```
# Output:
# Alice
# Bob
# Charlie
# All keys printed.
```

Iterating Over a Dictionary's Keys and Values

```
fruit_prices = {"Apple": 1.5, "Banana": 0.75, "Cherry": 2.0}

keys = list(fruit_prices.keys())

index = 0

while index < len(keys):
    key = keys[index]
    value = fruit_prices[key]
    print(f"{key}: ${value}")
    index += 1

print("All fruits and prices printed.")
```

```
# Output:
# Apple: $1.5
# Banana: $0.75
# Cherry: $2.0
# All fruits and prices printed.
```

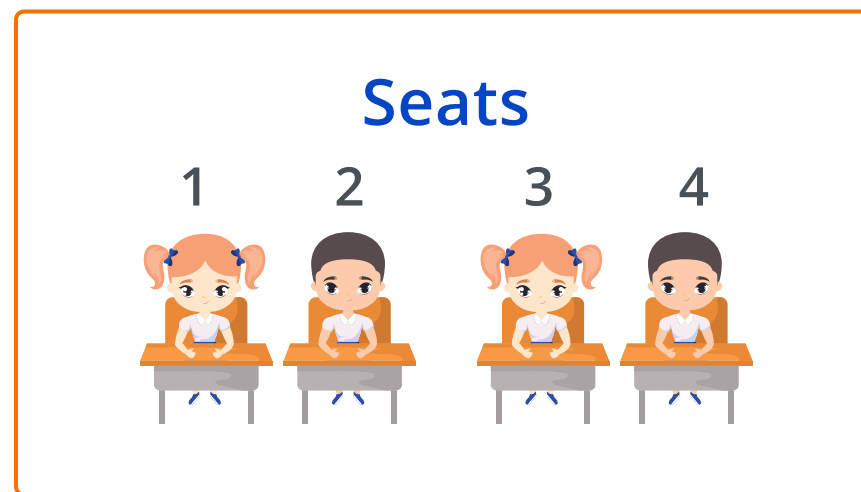
Nested loops

- What are Nested Loops?
- Basic Syntax
- Nested for Loop
- Nested while Loop

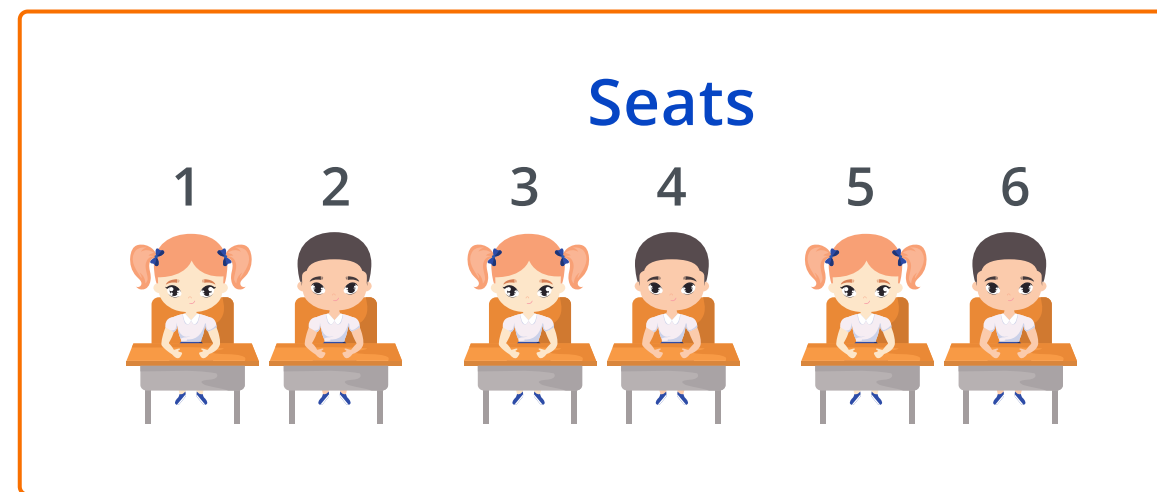
What Are Nested Loops?

- Nested loops mean **placing one loop inside another**.
- This lets you handle situations where you **need to repeat a set of actions within another set of actions**.

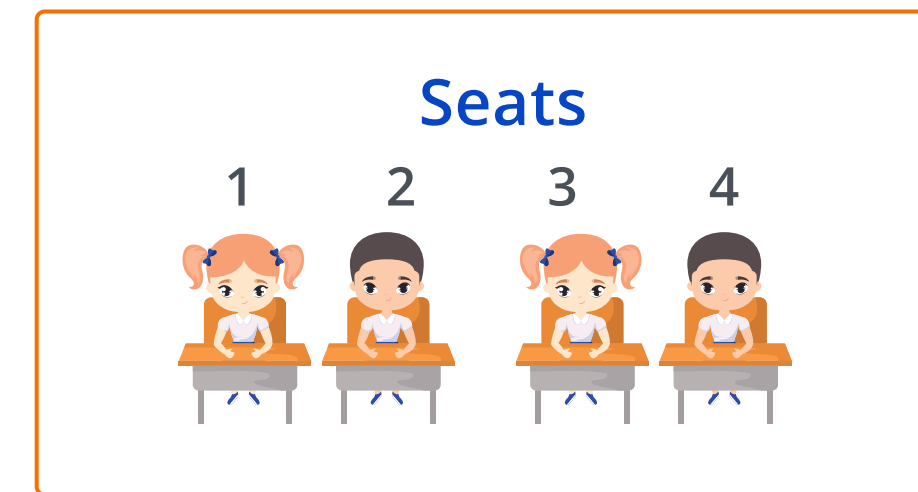
Think about checking classrooms in a school:



Class 1



Class 2

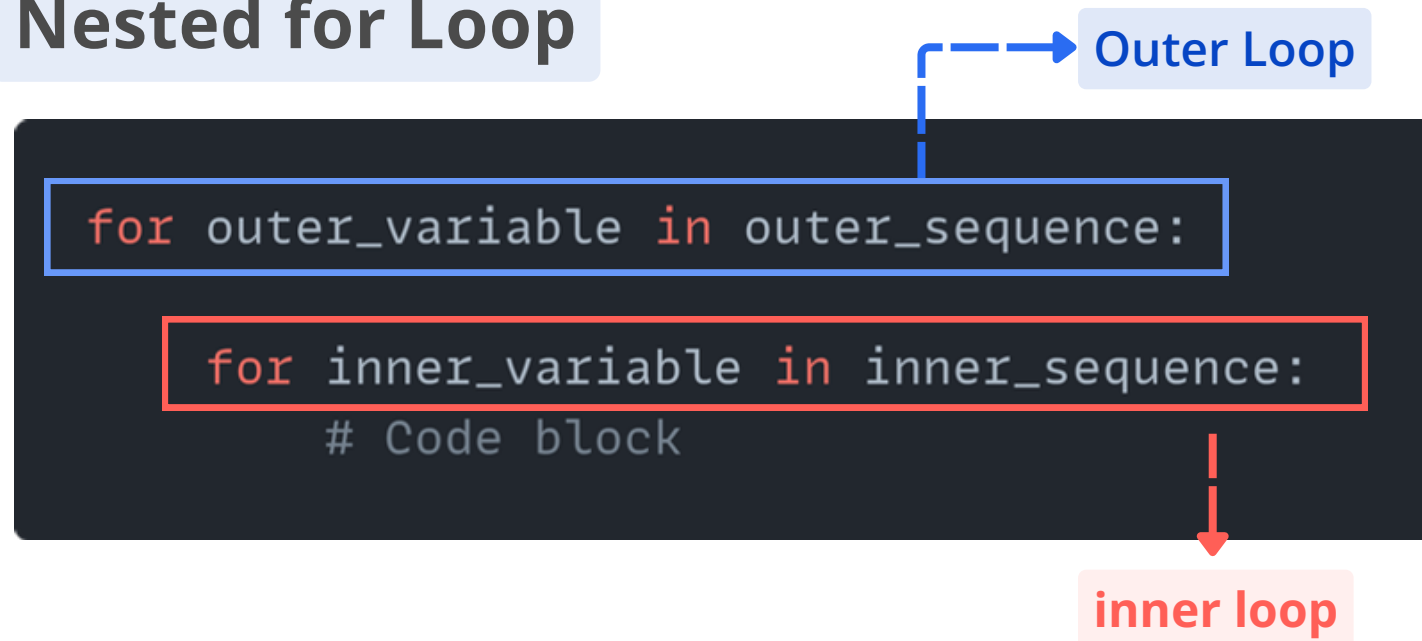


class 3

- **outer loop** → First, you go into each room one by one
- **inner loop** → Inside each room, you check each seat
- For each class, go through each seat.

Basic Syntax

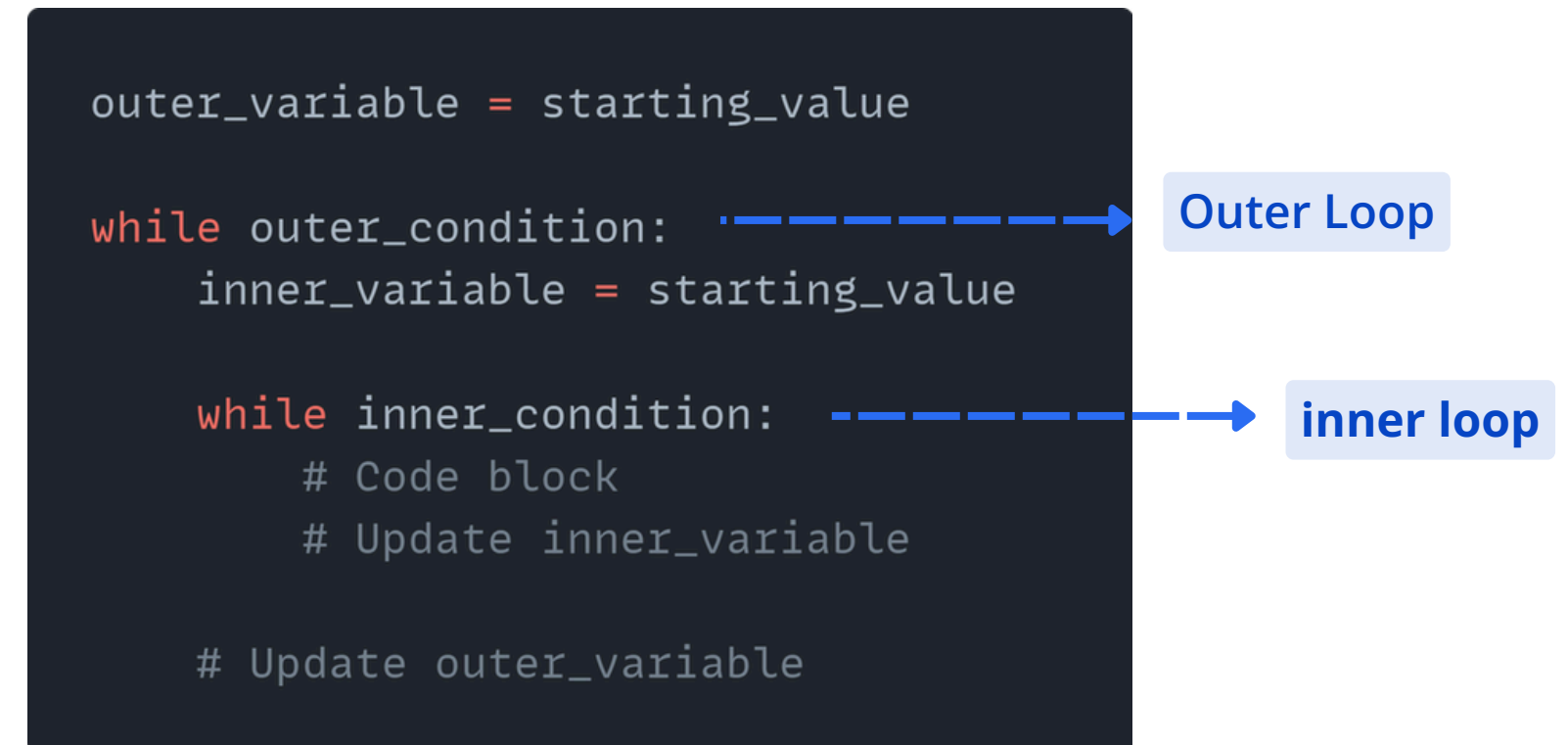
Nested for Loop



- **outer_variable:**
 - Takes each value from the `outer_sequence` one at a time
- **inner_variable:**
 - Iterates over the `inner_sequence` for each `outer_variable`

The inner loop runs completely for each value of the outer loop.

Nested while Loop



Nested for Loop

Multiplication Table

```
rows = [1, 2, 3] → Outer loop values  
cols = [1, 2, 3] → Inner loop values
```

```
for i in rows: →  
    for j in cols: →  
        print(f"{i} x {j} = {i * j}", end='\t')  
  
    print() → prints new line
```

The outer loop runs from 1 to 3 (i) → represents rows

The inner loop runs from 1 to 3 (j) → represents columns

end parameter → Print a tab instead of a new line

```
# 1 x 1 = 1    1 x 2 = 2    1 x 3 = 3  
# 2 x 1 = 2    2 x 2 = 4    2 x 3 = 6  
# 3 x 1 = 3    3 x 2 = 6    3 x 3 = 9
```

Iteration Breakdown

Outer Loop: i = 1

- **j = 1** → Output: **1 x 1 = 1**
- **j = 2** → Output: **1 x 2 = 2**
- **j = 3** → Output: **1 x 3 = 3**
- Inner loop ends → New line

Outer Loop: i = 2

- **j = 1** → Output: **2 x 1 = 2**
- **j = 2** → Output: **2 x 2 = 4**
- **j = 3** → Output: **2 x 3 = 6**
- Inner loop ends → New line

Outer Loop: i = 3

- **j = 1** → Output: **3 x 1 = 3**
- **j = 2** → Output: **3 x 2 = 6**
- **j = 3** → Output: **3 x 3 = 9**
- Inner loop ends → New line

Concept Practice Task

GamutX

Flattening a Matrix

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
flattened = []  
  
for row in matrix:  
    for item in row:  
        flattened.append(item)  
  
print(flattened)
```

```
# Output:  
# [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Nested while Loop

2x2 Grid of "X"

```
i = 1
while i <= 2:
    j = 1
    while j <= 2:
        print("X", end=' ')
        j += 1
    print()
    i += 1
```

```
# X X
# X X
```

- For each **cell in the grid**, it prints "X"
- After **each row**, it prints a new line using print()

Iteration Breakdown

Outer Loop (i = 1)

- **j = 1** → Output: **X**
- **j = 2** → Output: **X**
- Inner loop ends → **new line**
- **Output: X X**



Outer Loop (i = 2)

- **j = 1** → Output: **X**
- **j = 2** → Output: **X**
- Inner loop ends → **new line**
- **Output: X X**

Concept Practice Task

GamutX

Generating a Multiplication Table

```
size = 4

i = 1
while i <= size:
    j = 1
    while j <= size:
        print(i * j, end='\t')
        j += 1
    print()
    i += 1
```

Output:

# 1	2	3	4
# 2	4	6	8
# 3	6	9	12
# 4	8	12	16

Printing an Inverted Right-Angle Triangle

```
height = 5

i = height
while i > 0:
    j = 0
    while j < i:
        print('*', end=' ')
        j += 1
    print()
    i -= 1
```

Output:

#	*	*	*	*	*
#	*	*	*	*	
#	*	*	*		
#	*	*			
#	*				

Concept Practice Task

GamutX

Printing a Number Pattern

```
rows = 5

current_row = 1

while current_row <= rows:
    current_col = 1

    while current_col <= current_row:
        print(current_col, end=' ')
        current_col += 1

    print()
    current_row += 1
print("All rows printed.")
```

```
# Output:
# 1
# 1 2
# 1 2 3
# 1 2 3 4
# 1 2 3 4 5
# All rows printed.
```

Generating Combinations of Two Lists

```
fruits = ["Apple", "Banana", "Cherry"]
colors = ["Red", "Yellow", "Pink"]

fruit_index = 0

while fruit_index < len(fruits):
    color_index = 0

    while color_index < len(colors):
        print(f"{fruits[fruit_index]} - {colors[color_index]}")
        color_index += 1

    fruit_index += 1

print("All combinations printed.")
```

```
# Output:
# Apple - Red
# Apple - Yellow
# Apple - Pink
# Banana - Red
# Banana - Yellow
# Banana - Pink
# Cherry - Red
# Cherry - Yellow
# Cherry - Pink
# All combinations printed.
```

Loop Control Statements

- What Are Loop Control Statements?
- break - Statement
- continue - Statement
- pass - Statement

What Are Loop Control Statements?

- Loop control statements are **special commands that help you manage the flow inside loops.**
- They let you:
 - Stop a loop early,
 - Skip certain parts of the loop,
 - Or leave placeholders for code you'll add later.

Types of Loop Control Statements

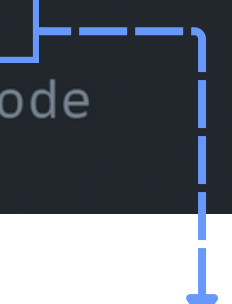
- **break** → Stops the loop immediately
- **continue** → Skips the current loop iteration
- **pass** → Does nothing — used as a placeholder

break Statement

- The break statement is used to **exit a loop early, even if the loop condition is still True.**
- What happens:
 - Python sees break
 - Loop ends right there
 - Program continues after the loop

break in a while loop:

```
while condition:  
    if condition_to_break:  
        break  
    # other code
```



break in a for loop:

```
for item in iterable:  
    if condition_to_break:  
        break  
    # other code
```



Exit loop: Stops the loop immediately if condition_to_break is True.

Understanding - Breaking Out of a Loop Early

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print("Searching for number 5 in the list:")

for num in numbers:
    if num == 5:
        print(f"Number {num} found! Exiting the loop.")
        break
    print(f"Checked number: {num}")
```

```
# Searching for number 5 in the list:
# Checked number: 1
# Checked number: 2
# Checked number: 3
# Checked number: 4
# Number 5 found! Exiting the loop.
```

As soon as num is 5, break is triggered.

Iteration Breakdown

Iteration 1

- num = 1
- if num == 5: → **False**
- Output: "Checked number: 1"

Iteration 3

- num = 3
- if num == 5: → **False**
- Output: "Checked number: 3"

Iteration 5

- num = 5
- if **num == 5**: → **True**
- Output: "Number 5 found! Exiting the loop."
- **break** → loop stops

Iteration 2

- num = 2
- if num == 5: → **False**
- Output: "Checked number: 2"

Iteration 4

- num = 4
- if num == 5: → **False**
- Output: "Checked number: 4"

Concept Practice Task

Finding the First Negative Number

```
numbers = [3, 7, 2, -5, 8, -2]

for num in numbers:
    if num < 0:
        print("First negative number is:", num)
        break
```

```
# Output:
# First negative number is: -5
```

First Item with a Value Greater Than 50 in a Dict

```
scores = {
    'John': 45,
    'Emily': 67,
    'Mike': 52,
    'Anna': 43
}

for name, score in scores.items():
    if score > 50:
        print("The first score greater than 50 is by:", name)
        break
```

```
# Output: The first score greater than 50 is by: Emily
```

Searching for a Specific Word in a List

```
words = ["apple", "banana", "cherry", "date"]

for word in words:
    if word == "cherry":
        print("Found the word 'cherry'.")
        break
```

```
# Output: Found the word 'cherry'.
```

First Even Number Greater Than 10

```
numbers = [12, 15, 22, 7, 18]

for num in numbers:
    if num > 10 and num % 2 == 0:
        print("First even number greater than 10 is:", num)
        break
```

```
# Output: First even number greater than 10 is: 12
```


Concept Practice Task

Finding the First Multiple of 7

```
number = 1

while number < 50:
    if number % 7 == 0:
        print("The first multiple of 7 less than 50 is:", number)
        break
    number += 1
```

```
# Output: The first multiple of 7 less than 50 is: 7
```

Counting Down to Zero

```
count = 10

while count >= 0:
    print("Count is:", count)
    if count == 0:
        break
    count -= 1
```

Output:

```
# Output:
# Count is: 10
# Count is: 9
# Count is: 8
# Count is: 7
# Count is: 6
# Count is: 5
# Count is: 4
# Count is: 3
# Count is: 2
# Count is: 1
# Count is: 0
```

Simulating a Simple Login System

```
correct_password = "secure123"
attempts = 0

while True:
    user_password = input("Enter your password: ")
    attempts += 1
    if user_password == correct_password:
        print("Login successful!")
        break
    elif attempts >= 3:
        print("Too many attempts. Access denied.")
        break
    else:
        print("Incorrect password. Please try again.")
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

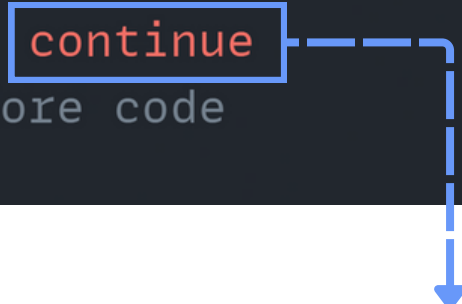
PS C:\Users\pralhad\Desktop\Work\Academy\gamutx-VSD-MASTER> & C:/Users/
p/Work/Academy/gamutx-VSD-MASTER/t.py
Enter your password: secure123
Login successful!
PS C:\Users\pralhad\Desktop\Work\Academy\gamutx-VSD-MASTER> |
```

continue - Statement

- The continue statement is used inside a loop when you want to:
- **Skip the rest of the current iteration**, and
- **Go straight to the next one.**

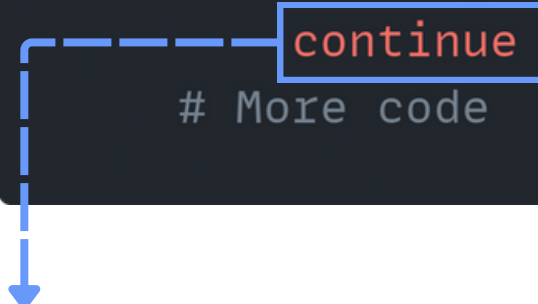
continue with a for loop

```
for item in iterable:  
    # Some code  
    if condition:  
        continue  
    # More code
```



continue with a while loop

```
while condition:  
    # Some code  
    if another_condition:  
        continue  
    # More code
```



Skips the rest of the current loop iteration and starts the next iteration of the loop.

Understanding - Skip Even Numbers

```
numbers = [1, 2, 3, 4, 5]

print("Skipping even numbers:")

for num in numbers:
    if num % 2 == 0:
        continue
    print(f"Odd number: {num}")
```

```
# Skipping even numbers:
# Odd number: 1
# Odd number: 3
# Odd number: 5
```

It skip even numbers

Iteration Breakdown

Iteration 1

- num = 1
- if num % 2 == 0: → **False**
- **Output: "Odd number: 1"**

Iteration 3

- num = 3
- if num % 2 == 0: → **False**
- **Output: "Odd number: 3"**

Iteration 5

- num = 5
- if num % 2 == 0: → **False**
- **Output: "Odd number: 5"**

Iteration 2

- num = 2
- if num % 2 == 0: → **True**
- **continue is executed → Skip rest loop**

Iteration 4

- num = 4
- if num % 2 == 0: → **True**
- **continue is executed → Skip rest loop**

Concept Practice Task

Skipping Negative Values in a List

```
numbers = [10, -5, 20, -2, 30]

for num in numbers:
    if num < 0:
        continue
    print("Positive number:", num)
```

Output:

- # Positive number: 10
- # Positive number: 20
- # Positive number: 30

Skipping Zero in a List of Numbers

```
numbers = [3, 0, 7, 0, 5]

for num in numbers:
    if num == 0:
        continue
    print("Non-zero number:", num)
```

Output:

- # Non-zero number: 3
- # Non-zero number: 7
- # Non-zero number: 5

Skipping Even Numbers in a Range

```
number = 1

while number <= 10:
    if number % 2 == 0:
        number += 1
        continue
    print("Odd number:", number)
    number += 1
```

----->

Output:

- # Odd number: 1
- # Odd number: 3
- # Odd number: 5
- # Odd number: 7
- # Odd number: 9

Concept Practice Task

Skipping Negative Numbers and Zero

```
count = -5

while count <= 5:
    if count <= 0:
        count += 1
        continue
    print("Positive number:", count)
    count += 1
```

```
# Output:
# Positive number: 1
# Positive number: 2
# Positive number: 3
# Positive number: 4
# Positive number: 5
```

Counting Vowel Occurrences in Dictionary Values

```
data = { 'first': 'apple', 'second': 'banana', 'third': 'cherry' }

vowel_counts = {'a': 0, 'e': 0, 'i': 0, 'o': 0, 'u': 0}

for value in data.values():
    index = 0
    while index < len(value):
        char = value[index]
        if char in vowel_counts:
            vowel_counts[char] += 1
        index += 1

print("Vowel counts:", vowel_counts)
```

```
# Output: Vowel counts: {'a': 4, 'e': 2, 'i': 0, 'o': 0, 'u': 0}
```

Skipping Characters in a String That Are Not Vowels

```
text = "Programming"
index = 0

while index < len(text):
    char = text[index]
    if char not in 'aeiouAEIOU':
        index += 1
        continue
    print("Vowel:", char)
    index += 1
```

```
# Output:
# vowel: o
# Vowel: o
# Vowel: i
```

Concept Practice Task

Skipping Specific Characters in a String

```
sentence = "Hello, World!"

for char in sentence:
    if char in "aeiou":
        continue
    print("Character:", char)
```

```
#Output:
Character: H
Character: l
Character: l
Character: ,
Character: 
Character: W
Character: r
Character: l
Character: d
Character: !
```

Skipping Characters Based on Manual Condition

```
text = "abc123def456ghi789"

for char in text:
    if char == '1' or char == '2' or char == '3':
        continue
    print("Character:", char)
```

```
#Output:
Character: a
Character: b
Character: c
Character: d
Character: e
Character: f
Character: 4
Character: 5
Character: 6
Character: g
Character: h
Character: i
Character: 7
Character: 8
Character: 9
```

pass Statement

- The **pass statement is a placeholder**. It's **used when**:
 - **Prevents syntax errors** when a code block is expected
 - Useful when you're **planning to add logic later**
 - **Keeps structure neat and clean**

🎯 Why Use pass?

- When you try to run following code

```
words = "Hello"  
for i in words:
```



```
PS C:\Users\HP\Desktop\tt> & "C:/Program Files/python.e  
File "c:\Users\HP\Desktop\tt\test.py", line 2  
    ^  
IndentationError: expected an indented block  
PS C:\Users\HP\Desktop\tt>
```

- It will give you an **error** because Python expects something inside the loop.

Basic Syntax

pass in a for loop:

```
for item in iterable:  
    pass
```

pass in a while loop:

```
while condition:  
    pass
```


Understanding - Using 'pass' in a loop

```
numbers = [1, 2, 3, 4, 5]

for num in numbers:
    if num % 2 == 0:
        pass
    else:
        print(f"Odd number: {num}")
```

```
# Odd number: 1
# Odd number: 3
# Odd number: 5
```

As soon as even number encounter it
do nothing

Iteration Breakdown

Iteration 1

- num = 1
- if num % 2 == 0: → **False**
- **Output: "Odd number: 1"**

Iteration 3

- num = 3
- if num % 2 == 0: → **False**
- **Output: "Odd number: 3"**

Iteration 5

- num = 5
- if num % 2 == 0: → **False**
- **Output: "Odd number: 5"**

Iteration 2

- num = 2
- if num % 2 == 0: → **True**
- **pass → Do nothing**

Iteration 4

- num = 4
- if num % 2 == 0: → **True**
- **pass → Do nothing**

Concept Practice Task

Using Pass in an Empty If Statement

```
value = 10

if value > 5:
    pass
else:
    print("Value is 5 or less.")

# Output:
# (No output, since the pass statement does nothing.)
```

Using Pass in a for Loop

```
numbers = [1, 2, 3, 4, 5]

for number in numbers:
    if number % 2 == 0:
        pass
    else:
        print(number)
```

```
# Output:
# 1
# 3
# 5
```

Using Pass in a While Loop

```
count = 5

while count > 0:
    if count == 3:
        pass
    else:
        print(count)
    count -= 1
```

```
# Output:
# 5
# 4
# 2
# 1
```

First Even Number Greater Than 10

```
inputs = ['text', 'number', 'date']

for input_type in inputs:
    if input_type == 'text':
        pass
    elif input_type == 'number':
        pass
    elif input_type == 'date':
        pass
print("All input types have been handled (implementation coming soon).")

# Output:
# All input types have been handled (implementation coming soon).
```

Concept Practice Task

Processing User Input Until a Valid Response is Given

```
valid_passwords = ['password123', 'admin2024', 'securePass!']
user_input = ''

while user_input not in valid_passwords:
    user_input = input("Enter your password: ")
    pass

print("Valid password entered.")
```

```
# Output:
# Enter your password: 123
# Enter your password: ferg
# Enter your password: password123
# Valid password entered.
```

Simulating a Countdown Timer

```
counter = 10

while counter > 0:
    pass

    counter -= 1

print("Countdown finished!")
```

```
# Output: Countdown finished!
```


Concept Practice Task

Implementing Role-Based Access Control

```
user_roles = {
    'Alice': 'admin',
    'Bob': 'editor',
    'Charlie': 'viewer'
}

for user, role in user_roles.items():
    if role == 'admin':
        # Placeholder for admin role handling
        pass # To be replaced with code for admin access
    elif role == 'editor':
        # Placeholder for editor role handling
        pass # To be replaced with code for editor access
    elif role == 'viewer':
        # Placeholder for viewer role handling
        pass # To be replaced with code for viewer access

print("User roles have been checked (implementation coming soon).")
# Output:
# User roles have been checked (implementation coming soon).
```

Common Mistakes in Loops

- Infinite Loops
- Incorrect loop range
- Forgetting to Update Loop Variables
- Using the Wrong Loop Type

Infinite Loops

- An infinite loop happens when the condition never becomes false, so the loop keeps running forever.

✗ Mistake:

```
while True:
    print("This will run forever!")
```

condition always true

- This will never stop unless manually interrupted (e.g., Ctrl + C)

✓ Best Practice:

```
count = 0
while count < 5:
    print("Count is", count)
    count += 1
```

help to break the loop

```
# Count is 0
# Count is 1
# Count is 2
# Count is 3
# Count is 4
```

- The loop ends once count reaches 5
- Always check that your loop condition can eventually become false!

Incorrect Loop Range

- Using the **wrong range values** can result in printing too many or too few items.

✗ Mistake:

```
for i in range(1, 6):  
    print(i)
```

```
# 1  
# 2  
# 3  
# 4  
# 5
```

- We wanted to start from 0 and print up to 5, this range is incorrect.

✓ Best Practice:

```
for i in range(6):  
    print(i)
```

```
# 0  
# 1  
# 2  
# 3  
# 4  
# 5
```

- range(stop) starts at 0 and goes up to (**but not including**) stop.

Forgetting to Update Loop Variables

- In a while loop, **if you forget to update the variable, the condition never changes**, and you get stuck in an **infinite loop**.

✗ Mistake:

```
i = 0
while i < 5:
    print(i)
```

- This leads to an **infinite loop** because i always stays at 0

✓ Best Practice:

```
i = 0
while i < 5:
    print(i)
    i += 1
```

```
# 0
# 1
# 2
# 3
# 4
```

- Always update the loop variable inside the loop.

Using the Wrong Loop Type

- Using a **while** loop when a **for** loop is more suitable can make your code less readable or longer than needed.

✗ Mistake:

```
count = 0
while count < 5:
    print(count)
    count += 1
```

```
# 0
# 1
# 2
# 3
# 4
```

Overcomplicating with while

✓ Best Practice:

```
for count in range(5):
    print(count)
```

```
# 0
# 1
# 2
# 3
# 4
```

- Use a **for** loop when you're working with a known range or going through items in a sequence.

The Purpose of Looping

Repetition

- We can **repeat actions multiple times without copying code**.

Automation

- Great for automating repetitive tasks — like **checking all items in a list or printing numbers**.

Efficiency

- Loops make your **code shorter, cleaner, and easier to update**.

Flexibility

- Loops can **adjust to different inputs or changing conditions while running**.

Thank You

